

FRIENDS & TEMPLATES

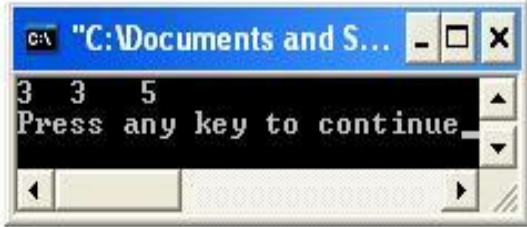
this Pointer

- Every object of a class maintains a (hidden) pointer to itself called `this`
- `this` is a reserved word
- When an object invokes a member function
 - the `this` pointer is referenced by the member function

this Pointer

```
class A
{
public:
    int x;
    int y;
    void fun() {cout<<x<<" "<<(*this).x<<" "<<this->y<<endl;}
};

void main()
{
    A obj1;
    obj1.x=3;
    obj1.y=5;
    obj1.fun();
}
```



```
class A
{
    int x;
public:
    A(){x=5;}
    A fun(){x=x+3; return *this;}
    void print(){cout<<x<<endl;}
};

void main()
{
    A obj1, obj2;
    obj1=obj2.fun();
    obj1.print();
    obj2.print();
}
```

8

8

```
test test::one()
{
    .
    .
    .
    return *this;
}
```

If x and y are objects of type test, then the statement:
`y = x.one();`

Copies the value
of object x into
object y

Friend Functions of Classes

- A **friend function** of a class is a nonmember function of the class, but has access to all the members (`public` or non-public) of the `class`.
- To make a function friend to a class
 - The reserved word `friend` precedes the function prototype in the class definition

Friend Functions of Classes (continued)

- The word `friend` appears only in the function prototype (in the `class` definition), not in the definition of the `friend` function
- When writing the `friend` function definition
 - The name of the `class` and the scope resolution operator are not used

Example

```
class Employee
{
    private:
        string name;
        double salary;
    public:
        Employee();
        Employee(string n, double s);
        string getName();
        double getSalary();
};

Employee::Employee() : name(""), salary(0) {}
Employee::Employee(string n, double s): name(n), salary(s) {}
string Employee::getName() { return name; }
double Employee::getSalary() { return salary; }
```

```
void raiseSalary(double amount, Employee& e)
{
    e.salary += amount;
}
```

Cannot access private
member of Employee

Example

```
class Employee
{
    private:
        string name;
        double salary;
    public:
        Employee();
        Employee(string n, double s);
        string getName();
        double getSalary();
        friend void raiseSalary(double amount, Employee& e);
};

Employee::Employee() : name(""), salary(0) {}
Employee::Employee(string n, double s): name(n), salary(s) {}
string Employee::getName() { return name; }
double Employee::getSalary() { return salary; }
```

```
void raiseSalary(double amount, Employee& e)
{
    e.salary += amount;
}
```

Can now access private
members of Employee

Friend Functions & Inheritance

```
class A
{
    friend void func1();
    int x;
};
```

```
class B: public A
{
    friend void func2();
    int y;
};
```

```
void func1()
{
    B obj;
    obj.x = 1;
    //obj.y = 2    cannot access, not a friend of B
}
```

```
void func2()
{
    B obj;
    //obj.x = 1;    cannot access, not a friend of A
    obj.y = 2;
}
```


Friend Classes

- A **friend class** is a class that can access the private and protected members of a class in which it is declared as **friend**.
- This is needed when we want to allow a particular class to access the **private** and **protected** members of a class.

Example

```
class Employee
{
    private:
        string name;
        double salary;
    public:
        Employee();
        Employee(string n, double s);
        string getName();
        double getSalary();
};

Employee::Employee() : name(""), salary(0) {}
Employee::Employee(string n, double s): name(n), salary(s) {}
string Employee::getName() { return name; }
double Employee::getSalary() { return salary; }
```

```
class Boss
{
    public:
        Boss();
        void giveRaise(double amount, Employee& e);
};

Boss::Boss() {}
void Boss::giveRaise(double amount, Employee& e)
{
    e.salary += amount;
}
```

Cannot access, private member of Employee

Example

```
class Employee
{
    private:
        string name;
        double salary;
    public:
        Employee();
        Employee(string n, double s);
        string getName();
        double getSalary();
        friend class Boss;
};
```

```
class Boss
{
    public:
        Boss();
        void giveRaise(double amount, Employee& e);
};

Boss::Boss() {}
void Boss::giveRaise(double amount, Employee& e)
{
    e.salary += amount;
}
```

Can access private and protected members of Employee

```
Employee::Employee() : name(""), salary(0) {}
Employee::Employee(string n, double s): name(n), salary(s) {}
string Employee::getName() { return name; }
double Employee::getSalary() { return salary; }
```

Templates

- Templates: a single code body for a set of related functions (called function template) and related classes (called class template)
- The syntax for templates is:

```
template <class Type>  
    declaration
```

where `Type` is the type of the data and declaration is either a function declaration or a class declaration

Templates (continued)

- `template` is a reserved word
- The word `class` in the heading refers to any user-defined type or built-in type
- `Type` is called a formal parameter to the template
- Just as variables are parameters to functions
 - Data types are parameters to templates

Function Templates

- The syntax of the function template is:

```
template <class Type>  
    function definition;
```

where `Type` is called a formal parameter of the template

- `Type`
 - Specifies type of parameters to the function
 - Specifies return type of the function
 - Declares variables within the function

```

#include <iostream>
#include<string>
using namespace std;
int fun(int a, int b)
{
    if (a>b)
        return a;
    else
        return b;
}

double fun(double a, double b)
{
    if (a>b)
        return a;
    else
        return b;
}

string fun(string a, string b)
{
    if (a>b)
        return a;
    else
        return b;
}

void main()
{
    cout<<fun(8,5)<<endl;
    cout<<fun("Ali","Ahmed")<<endl;
    cout<<fun(3.4, 5.5)<<endl;
}

```

```

#include <iostream>
#include<string>
using namespace std;

template <class T>

T fun(T a, T b)
{
    if (a>b)
        return a;
    else
        return b;
}

void main()
{
    cout<<fun(8,5)<<endl;
    cout<<fun("Ali","Ahmed")<<endl;
    cout<<fun(3.4, 5.5)<<endl;
}

```



```

C:\WINDOWS\system...
8
Ali
5.5
Press any key to continue

```

Example

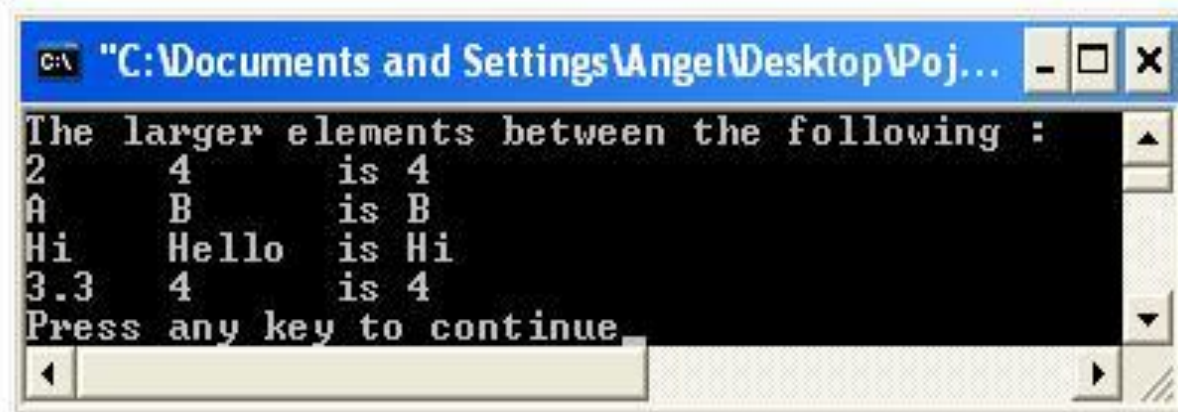
```
#include <iostream>
#include<string>
using namespace std;
```

```
template<class A>
A larger(A x, A y)
{
```

```
    if (x> y)
        return x;
    else
        return y;
}
```

```
void main()
{
```

```
    int a=2, b=4;
    char ch1='A', ch2= 'B';
    string s1="Hi", s2="Hello";
    double var1=3.3, var2=4;
    cout<<"The larger elements between the following :\n";
    cout<<a<<"    "<<b<<"    "    is "<<larger(a,b)<<endl;
    cout<<ch1<<"    "<<ch2<<"    "    is "<<larger(ch1,ch2)<<endl;
    cout<<s1<<"    "<<s2<<"    is "<<larger(s1,s2)<<endl;
    cout<<var1<<"    "<<var2<<"    "    is "<<larger(var1,var2)<<endl;
}
```



```
C:\Documents and Settings\Angel\Desktop\Poj...
The larger elements between the following :
2      4      is 4
A      B      is B
Hi     Hello  is Hi
3.3    4      is 4
Press any key to continue
```


Class Templates

- Class templates: a single code segment represents a set of related classes
- Syntax:

```
template <class Type>  
class declaration
```

- Called parameterized types
 - A specific class is made based on the parameter type

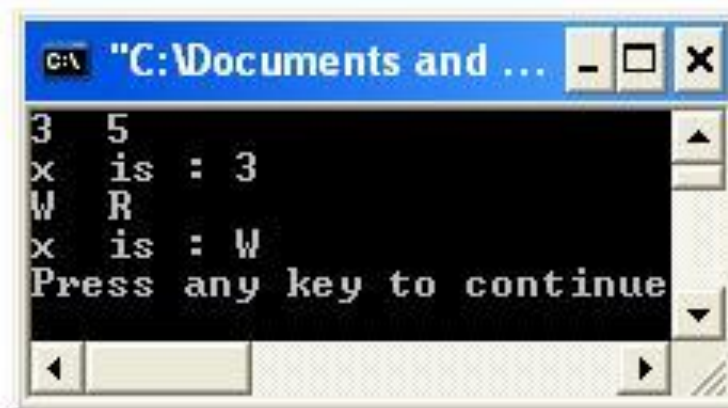
Example

```
template<class T>
class A
{
    T x;
    T y;
public:
    void set(T a, T b) {x=a; y=b;}
    void print(){cout<<x<<" "<<y<<endl;}
    T getX();
};

template<class T>
T A <T>::getX()
{
    return x;
}

void main()
{
    A<int> obj;
    obj.set(3,5);
    obj.print();
    cout<<"x is : "<<obj.getX()<<endl;

    A<char> obj2;
    obj2.set('W','R');
    obj2.print();
    cout<<"x is : "<<obj2.getX()<<endl;
}
```



THANKS
